

[پروژه پایان ترم معماری کامپیوتر، CPULator]

| ریحانه داداش پور، ۴۰۲۲۴۳۰۶۰

| نازنین فروتن، ۴۰۲۲۴۳۰۸۴

| ترم بهار ۴۰۴

| استاد چشمی خانی

برای این پروژه از چند بخش داده برای مدیریت اطلاعات استفاده می کنیم:

- **Instruction**: فضای ۶۴ بایتی برای ذخیره کردن هر خط دستور ورودی.
- **Debt**: فضای ۴۰۰۰۰ (یعنی $100 * 100 * 4$) (مقدار لازم برای ذخیره هر integer) بایتی برای ذخیره اطلاعات بدهی افراد در یک آرایه دوبعدی. مقدار `debt[i][j]` نشان دهنده مبلغی است که فرد `i` به فرد `j` بدهکار است.
- **Names**: فضای ۴۰۰ بایتی برای ذخیره اسم حداکثر ۱۰۰ نفر که هر کدام از اسامی می تواند حداکثر ۸ کاراکتر باشد.
- **Benefit**: فضای ۴۰۰ بایتی برای ذخیره سرمایه خالصی که هر فرد دارد.
- **People_count**: برای ذخیره تعداد کل افراد متمایز.
- **Buffer**: فضای ۳۲ بایتی که به عنوان بافر موقت برای ذخیره اسامی و مقدار پول از خط دستور استفاده می شود.

روند برنامه:

برنامه با خواندن تعداد دستورات شروع می شود و سپس وارد یک حلقه اصلی می شود که هر دستور را یکی یکی پردازش می کند. هر کدام از دستورات در بافر `instruction` خوانده می شود که اولین کارکتر آن برای تعیین نوع دستور استفاده می شود. دستورات به شرح زیر هستند:

1 - ثبت کردن قرض و بدهی

ورودی به صورت `<amount> <name2> <name1>` 1 می‌باشد. که قرض `name1` به `name2` و به مبلغ `amount` است. ابتدا با استفاده از تابع `find_name_index` بررسی می‌شود که این دو شخص از قبل وجود دارند یا نه. اگر اسمی جدید باشد، به آرایه `names` اضافه می‌شود و همچنین مقدار `people_count` افزایش می‌یابد.

مقدار `amount` با استفاده از تابع `parse_money` به صورت یک عدد صحیح با ۲ رقم اعشار تجزیه می‌شود و سپس ماتریس `debt` و سرمایه خالص هر فرد (`benefit`) آپدیت می‌شوند.

2 - پیدا کردن ثروتمندترین فرد

ورودی آن عدد ۲ است و اسم شخصی که بیشترین سرمایه خالص مثبت را دارد چاپ می‌کند. از طریق آرایه `benefit` برنامه تکرار می‌شود تا حداکثر مقدار پیدا شود. اگر چند نفر همزمان دارای حداکثر مقدار سرمایه بودند، براساس ترتیب لغتنامه‌ای (با تابع `lexical_compare`) کسی که کوچکتر بود ثروتمندترین شناخته می‌شود. اگر هم هیچکس سرمایه خالص مثبت نداشت مقدار -۱ چاپ می‌شود.

3 - پیدا کردن فقیرترین فرد

ورودی آن عدد ۳ است و مشابه توابع ثروتمندترین فرد، برای پیدا کردن فقیرترین شخص کار می‌کند.

4 - تعداد افراد بدهکار به شخص s

ورودی آن به صورت `<name>` 4 است و تعداد افرادی که به `name` بدهکار هستند را شمرده و چاپ می‌کند.

از طریق سطر ماتریس `debt` ورودی‌هایی که بدهی آن‌ها بزرگتر از ۰ است را می‌شمارد. (با تابع `calculate_debt`)

5 - تعداد افراد طلبکار از شخص s

ورودی آن به صورت `<name>` 5 است و تعداد افرادی که `name` از آن‌ها طلبکار است را خروجی می‌دهد. از طریق ستون ماتریس `debt` مربوط به `name` تکرار می‌کند و ورودی‌هایی که بدهی آن‌ها بزرگتر از ۰ است را می‌شمارد. (با تابع `calculate_debt`)

6 - محاسبه بدهی خالص شخص s1 به s2

ورودی آن به صورت `<name1> <name2>` 6 است و مبلغی را که `name1` باید به `name2` بدهد تا بی حساب شوند را محاسبه و چاپ می‌کند.

توابع کلیدی برنامه:

- **Parse_instruction**: شماره دستور، اسامی و مقدار پول قرض داده شده را از رشته ورودی جدا کرده و در buffer ذخیره می کند.
- **Empty_memory**: تابعی است برای صفر کردن یک بلوک حافظه که برای مقداردهی اولیه دستورات و بافر استفاده می شود.
- **Add_new_name**: یک اسم جدید را به آرایه names اضافه می کند و همچنین people_count را زیاد می کند.
- **Find_name_index**: آرایه names را می گردد تا اسم داده شده را پیدا کند و index آن را برمی گرداند. اگر پیدا نشد -۱ برمی گرداند.
- **String_compare (strcmp)**: دو رشته حداکثر ۸ کاراکتری را مقایسه می کند و در صورت برابر بودن ۱ و در غیر این صورت ۰ را برمی گرداند.
- **Lexical_compare**: برای مقایسه براساس ترتیب لغتنامه ای استفاده می شود و اسم شخصی که از نظر الفبایی کوچکتر است را برمی گرداند.
- **Parse_money**: یک عدد اعشاری برای مقدار پول را به عدد صحیح تبدیل می کند تا محاسبات آسان تر شود. (از بخش صحیح و اعشاری نیز نگهداری می کند).
- **Calculate_debt**: براسا ماتریس debt، بدهی خالص بین دو شخص را محاسبه می کند.